SQL ASSIGNMENT

Group By, Nested Subquery and Aggregate Functions

Assumed database structure

The assignment does not specify table definitions. The following assumed tables and columns are used in the solutions:

- **Department** (DeptID, DeptName, HOD_EmpID)
- **Faculty** (EmpID, Name, Salary, DeptID, SupervisorID)
- **Course** (CourseID, CourseName, DegreeType, DeptID)
- **Subject** (SubjectID, SubjectName, CourseID)
- **Student** (RollNo, Name)
- **Semester** (SemesterID, SemesterNo, Session)
- **Enrollment** (RollNo, CourseID, SemesterID)
- **Marks** (RollNo, SubjectID, SemesterID, Marks)

Note: These are assumed structures. Change table or column names to match your actual database. The ranking/CTE queries require MySQL 8.0 or later.

(a) Maximum and minimum salary of departments whose average salary is at least Rs. 10,000

SQL Query:

```
SELECT
    DeptID,
    MAX(Salary) AS Maximum_Salary,
    MIN(Salary) AS Minimum_Salary,
    AVG(Salary) AS Average_Salary
FROM Faculty
GROUP BY DeptID
HAVING AVG(Salary) >= 10000;
```

Explanation:

- GROUP BY DeptID groups faculty by department.
- MAX(Salary) and MIN(Salary) find the highest and lowest salaries.
- AVG(Salary) calculates the departmental average.
- HAVING keeps only departments whose average salary is at least 10,000.

(b) Average salary of departments offering both B.Tech and M.Tech and having at least 2 faculty members

SQL Query:

```
SELECT DeptID, AVG(Salary) AS Average_Salary
FROM Faculty
WHERE DeptID IN
(
    SELECT DeptID
    FROM Course
    WHERE DegreeType IN ('B.Tech', 'M.Tech')
    GROUP BY DeptID
    HAVING COUNT(DISTINCT DegreeType) = 2
)
GROUP BY DeptID
HAVING COUNT(EmpID) >= 2;
```

Explanation:

- The inner query finds departments offering both degree types.

- COUNT(DISTINCT DegreeType) = 2 ensures that B.Tech and M.Tech are both present.
- The outer query calculates average faculty salary and requires at least two faculty members.

(c) Average salary of qualifying faculty in departments offering both degrees, with at least 2 faculty paid above their departmental average

SQL Query:

```
SELECT DeptID, AVG(Salary) AS Average_Salary
FROM Faculty
WHERE DeptID IN
(
    SELECT DeptID
    FROM Course
    WHERE DegreeType IN ('B.Tech', 'M.Tech')
    GROUP BY DeptID
    HAVING COUNT(DISTINCT DegreeType) = 2
)
AND Salary >
(
    SELECT AVG(F2.Salary)
    FROM Faculty F2
    WHERE F2.DeptID = Faculty.DeptID
)
GROUP BY DeptID
HAVING COUNT(EmpID) >= 2;
```

Explanation:

- The first subquery identifies departments offering both B.Tech and M.Tech.
- The correlated subquery calculates the average salary in the current faculty member's department.
- Only faculty earning more than their departmental average are retained.
- HAVING ensures at least two such faculty members exist; AVG then averages the selected faculty salaries.

(d)(i) Supervisors managing at least 2 faculty members within their own department

SQL Query:

```
SELECT
    S.EmpID,
    S.Name,
    S.Salary
FROM Faculty S
JOIN Faculty F
    ON S.EmpID = F.SupervisorID
   AND S.DeptID = F.DeptID
GROUP BY S.EmpID, S.Name, S.Salary
HAVING COUNT(F.EmpID) >= 2;
```

Explanation:

- S represents the supervisor and F represents supervised faculty.
- The join matches employees to their supervisor and restricts them to the same department.
- HAVING COUNT(F.EmpID) >= 2 selects supervisors with at least two such employees.

(d)(ii) Supervisors managing at most 2 faculty members outside their own department

SQL Query:

```
SELECT
    S.EmpID,
    S.Name,
    S.Salary
FROM Faculty S
LEFT JOIN Faculty F
    ON S.EmpID = F.SupervisorID
   AND S.DeptID <> F.DeptID
```

```
GROUP BY S.EmpID, S.Name, S.Salary
HAVING COUNT(F.EmpID) <= 2;
```

Explanation:

- LEFT JOIN retains supervisors even when they supervise no one outside their department.
- S.DeptID <> F.DeptID matches supervised faculty from other departments.
- HAVING COUNT(F.EmpID) <= 2 selects supervisors with at most two such employees.

(e)(i) HODs of departments offering either B.Tech or M.Tech, but not both

SQL Query:

```
SELECT DISTINCT
    F.EmpID,
    F.Name,
    F.Salary
FROM Faculty F
JOIN Department D
    ON F.EmpID = D.HOD_EmpID
WHERE D.DeptID IN
(
    SELECT DeptID
    FROM Course
    GROUP BY DeptID
    HAVING COUNT(DISTINCT DegreeType) = 1
       AND MAX(DegreeType) IN ('B.Tech', 'M.Tech')
);
```

Explanation:

- The subquery groups courses by department.
- COUNT(DISTINCT DegreeType) = 1 ensures only one degree type is present.
- MAX(DegreeType) restricts the result to B.Tech or M.Tech.
- The outer query returns the corresponding HOD's details.

(e)(ii) HODs of departments offering at least two B.Tech courses or at least two M.Tech courses

SQL Query:

```
SELECT DISTINCT
    F.EmpID,
    F.Name,
    F.Salary
FROM Faculty F
JOIN Department D
    ON F.EmpID = D.HOD_EmpID
WHERE D.DeptID IN
(
    SELECT DeptID
    FROM Course
    GROUP BY DeptID
    HAVING
        SUM(CASE
            WHEN DegreeType = 'B.Tech' THEN 1
            ELSE 0
        END) >= 2
        OR
        SUM(CASE
            WHEN DegreeType = 'M.Tech' THEN 1
            ELSE 0
        END) >= 2
);
```

Explanation:

- CASE counts B.Tech and M.Tech courses separately.

- SUM adds the counts for each degree type.
- OR allows either of the two conditions to qualify the department.
- The outer query displays the HOD's ID, name and salary.

(e)(iii) HODs of departments having a course with a subject name containing the word 'machine'

SQL Query:

```
SELECT DISTINCT
    F.EmpID,
    F.Name,
    F.Salary
FROM Faculty F
JOIN Department D
    ON F.EmpID = D.HOD_EmpID
WHERE EXISTS
(
    SELECT 1
    FROM Course C
    JOIN Subject S
        ON C.CourseID = S.CourseID
    WHERE C.DeptID = D.DeptID
      AND LOWER(S.SubjectName) LIKE '%machine%'
);
```

Explanation:

- EXISTS checks whether at least one matching subject is present.
- The join connects courses with their subjects.
- LIKE '%machine%' searches for the word anywhere in the subject name.
- LOWER makes the search case-insensitive in common MySQL configurations.

(f) Aggregate marks of each student in B.Tech CSE Semester 5 for the current session

SQL Query:

```
SELECT
    ST.RollNo,
    ST.Name,
    SUM(M.Marks) AS Aggregate_Marks
FROM Student ST
JOIN Enrollment E
    ON ST.RollNo = E.RollNo
JOIN Course C
    ON E.CourseID = C.CourseID
JOIN Semester SEM
    ON E.SemesterID = SEM.SemesterID
JOIN Marks M
    ON ST.RollNo = M.RollNo
   AND SEM.SemesterID = M.SemesterID
JOIN Subject SUB
    ON M.SubjectID = SUB.SubjectID
   AND SUB.CourseID = C.CourseID
WHERE C.CourseName = 'CSE'
  AND C.DegreeType = 'B.Tech'
  AND SEM.SemesterNo = 5
  AND SEM.Session = '2026-27'
GROUP BY ST.RollNo, ST.Name;
```

Explanation:

- Joins connect students, enrollment, course, semester, marks and subjects.
- The WHERE clause selects B.Tech CSE Semester 5 for the specified session.
- SUM(M.Marks) calculates each student's aggregate marks.
- GROUP BY produces one result per student.

- Replace '2026-27' with the session value used in your database.

(g) Topper students of each course in each semester

SQL Query:

```
WITH Student_Total AS
(
    SELECT
        ST.RollNo,
        ST.Name,
        C.CourseID,
        C.CourseName,
        SEM.SemesterID,
        SEM.SemesterNo,
        SUM(M.Marks) AS Total_Marks
    FROM Student ST
    JOIN Enrollment E
        ON ST.RollNo = E.RollNo
    JOIN Course C
        ON E.CourseID = C.CourseID
    JOIN Semester SEM
        ON E.SemesterID = SEM.SemesterID
    JOIN Marks M
        ON ST.RollNo = M.RollNo
       AND SEM.SemesterID = M.SemesterID
    JOIN Subject SUB
        ON M.SubjectID = SUB.SubjectID
       AND SUB.CourseID = C.CourseID
    GROUP BY
        ST.RollNo,
        ST.Name,
        C.CourseID,
        C.CourseName,
        SEM.SemesterID,
        SEM.SemesterNo
),
Ranked_Students AS
(
    SELECT *,
        RANK() OVER
        (
            PARTITION BY CourseID, SemesterID
            ORDER BY Total_Marks DESC
        ) AS Student_Rank
    FROM Student_Total
)
SELECT
    RollNo,
    Name,
    CourseName,
    SemesterNo,
    Total_Marks
FROM Ranked_Students
WHERE Student_Rank = 1;
```

Explanation:

- The first CTE calculates each student's total marks for a course and semester.
- RANK assigns positions according to total marks.
- PARTITION BY creates a separate ranking for every course and semester.
- Student_Rank = 1 returns the topper; ties are included.

(h) Personal details of the 4th highest-paid faculty employee

SQL Query:

```
WITH Salary_Rank AS
(
    SELECT
        EmpID,
        Name,
        Salary,
```

```
        DENSE_RANK() OVER
        (
            ORDER BY Salary DESC
        ) AS R
    FROM Faculty
)
SELECT EmpID, Name, Salary
FROM Salary_Rank
WHERE R = 4;
```

Explanation:

- ORDER BY Salary DESC sorts salaries from highest to lowest.
- DENSE_RANK assigns the same rank to tied salaries without gaps.
- R = 4 returns employees receiving the fourth-highest distinct salary.

(i) Personal details of the 4th highest-paid and 4th lowest-paid faculty employees

SQL Query:

```
WITH Salary_Rank AS
(
    SELECT
        EmpID,
        Name,
        Salary,
        DENSE_RANK() OVER
        (
            ORDER BY Salary DESC
        ) AS Highest_Rank,
        DENSE_RANK() OVER
        (
            ORDER BY Salary ASC
        ) AS Lowest_Rank
    FROM Faculty
)
SELECT
    EmpID,
    Name,
    Salary,
    CASE
        WHEN Highest_Rank = 4
            THEN '4th Highest'
        WHEN Lowest_Rank = 4
            THEN '4th Lowest'
    END AS Salary_Category
FROM Salary_Rank
WHERE Highest_Rank = 4
   OR Lowest_Rank = 4;
```

Explanation:

- The first DENSE_RANK ranks salaries from highest to lowest.
- The second DENSE_RANK ranks salaries from lowest to highest.
- CASE labels each selected employee as 4th Highest or 4th Lowest.
- The WHERE clause returns employees satisfying either rank.

(j) Personal details of the 3rd highest-paid faculty employee in each department

SQL Query:

```
WITH Department_Salary AS
(
    SELECT
        EmpID,
        Name,
        Salary,
        DeptID,
        DENSE_RANK() OVER
        (
```

```
            PARTITION BY DeptID
            ORDER BY Salary DESC
        ) AS Salary_Rank
    FROM Faculty
)
SELECT
    EmpID,
    Name,
    Salary,
    DeptID
FROM Department_Salary
WHERE Salary_Rank = 3;
```

Explanation:

- PARTITION BY DeptID creates a separate ranking for each department.
- ORDER BY Salary DESC ranks faculty from highest salary to lowest.
- DENSE_RANK assigns ranks within each department.
- Salary_Rank = 3 returns the third-highest distinct salary in each department; ties are included.

Important SQL functions and keywords

- **GROUP BY:** Groups records.
- **HAVING:** Filters grouped records.
- **AVG():** Calculates an average.
- **MAX() / MIN():** Find maximum / minimum values.
- **SUM():** Calculates a total.
- **COUNT():** Counts records.
- **DISTINCT:** Removes duplicate values.
- **EXISTS:** Checks whether matching records exist.
- **LIKE:** Performs pattern matching.
- **CASE:** Performs conditional operations.
- **DENSE_RANK():** Assigns ranks without gaps.
- **PARTITION BY:** Creates separate ranking groups.
- **WITH:** Defines a Common Table Expression (CTE).

Final note: *These solutions use the assumed schema stated at the beginning. Verify the actual table and column names before execution. Questions (g), (h), (i) and (j) use CTEs and window functions supported by MySQL 8.0 and later.*